

WYKŁAD 8

Pliki, I/O i protokół iteracji

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

1	Operacje na plikach	<code>open()</code> , tryby, odczyt i zapis, kodowanie
2	Menedżer kontekstu (with)	bezpieczne I/O, własny <code>__enter__</code> / <code>__exit__</code>
3	Pliki binarne i pathlib	bajty, struct, nowoczesne ścieżki
4	CSV i JSON	<code>csv.reader/writer</code> , <code>DictReader</code> , <code>json.dumps/loads</code>
5	Protokół iteracji	<code>__iter__</code> i <code>__next__</code> , <code>StopIteration</code>
6	Generatory i yield	<code>yield</code> , wyrażenia generatorowe, <code>send()</code>
7	yield from	delegacja do subgeneratora
8	itertools zaawansowane	<code>chain</code> , <code>groupby</code> , <code>product</code> , <code>starmap</code> , <code>tee</code>
9	contextlib	<code>contextmanager</code> , <code>suppress</code> , <code>ExitStack</code>

Operacje na plikach

`open()`, tryby dostępu, odczyt i zapis

Funkcja open()

Podstawowa brama do systemu plików

Sygnatura pełna:

```
f = open('dane.txt', 'r',  
        encoding='utf-8')  
content = f.read()  
print(content)  
f.close() # ZAWSZE zamknąć!  
  
open(file, mode='r',  
      buffering=-1,  
      encoding=None,  
      errors=None,  
      newline=None,  
      closefd=True)
```

Najważniejsze tryby dostępu:

'r'	odczyt (domyślny)	'w'	zapis (nadpisuje!)
'a'	dopisywanie na końcu	'x'	tworzenie (błąd jeśli istnieje)
'b'	tryb binarny	't'	tryb tekstowy (domyślny)
'r+'	odczyt + zapis	'rb'	binarny odczyt

Odczyt pliku — metody

<code>read()</code>	Cała zawartość jako jeden string
<code>read(n)</code>	Odczytaj n znaków (lub bajtów w trybie binarnym)
<code>readline()</code>	Jedna linia (z <code>\n</code> na końcu)
<code>readlines()</code>	Lista wszystkich linii
<code>for line in f:</code>	najlepsza metoda — leniwa iteracja

Przykład iteracji po liniach (zalecany sposób):

```
with open('dane.txt') as f:
    for line in f:
        print(line.strip())
```

✓ Iteracja linia po linii nie wczytuje całego pliku do pamięci — idealne dla dużych plików.

Zapis do pliku

```
# Zapis – tryb 'w' (nadpisuje!)
with open('out.txt', 'w') as f:
    f.write('Linia 1\n')
    f.write('Linia 2\n')

# print() też może pisać do pliku
print('Linia 3', file=f)

# Dopisywanie – tryb 'a'
with open('log.txt', 'a') as f:
    f.write('Nowy wpis\n')

# writelines() – lista stringów
lines = ['AAA\n', 'BBB\n']
with open('out.txt', 'w') as f:
    f.writelines(lines)
```

Przekierowanie sys.stdout:

```
import sys
with open('output.txt', 'w') as f:
    sys.stdout = f
    print('To trafi do pliku!')
sys.stdout = sys.__stdout__
```

⚠ Tryb 'w' NADPISUJE istniejący plik bez ostrzeżenia! Jeśli chcesz dodać na końcu — użyj 'a'.
Jeśli chcesz zapobiec nadpisaniu — użyj 'x' (exclusive creation).

Instrukcja with — bezpieczeństwo I/O

Źle:

```
f = open('dane.txt')
try:
    data = f.read()
finally:
    f.close() # łatwo zapomnieć!
```

Dobrze:

```
with open('dane.txt') as f:
    data = f.read()
# f.close() wywołane
# automatycznie!
```

Jak działa with?

```
class MojPlik:
    def __init__(self, path):
        self.f = open(path)
    def __enter__(self):
        return self.f
    def __exit__(self, *args):
        self.f.close()
        return False
```

Protokół context managera:

1. `__enter__()` → zwraca zasób
2. Blok `with` się wykonuje
3. `__exit__()` → sprzątanie

Jeśli `__exit__` zwraca `True`,
wyjątek jest połknięty
(rzadko używane).

Pliki binarne

Tryb 'rb' / 'wb' — bajty zamiast tekstu

```
# Odczyt binarny
with open('obraz.png', 'rb') as f:
    header = f.read(8)
    print(header) # b'\x89PNG...'
    print(type(header)) # <bytes>

# Zapis binarny
data = bytes([0x48, 0x65, 0x6C])
with open('out.bin', 'wb') as f:
    f.write(data)
    f.write(b'\x00\xff')
```

Moduł struct — pakowanie danych:

```
import struct
# Pakowanie: int(4B)+float(4B)+char
packed = struct.pack('if1s',
                     42, 3.14, b'A')
with open('data.bin', 'wb') as f:
    f.write(packed)
```

'i'	→ int (4B)	'f'	→ float (4B)	'd'	→ double (8B)
's'	→ char[] (bajty)	'?'	→ bool (1B)		

pathlib — nowoczesne ścieżki

Moduł pathlib (Python 3.4+) zastępuje os.path

```
from pathlib import Path

p = Path('projekt')/'dane'/'plik.csv'
print(p)          # projekt/dane/plik.csv
print(p.name)     # plik.csv
print(p.stem)     # plik
print(p.suffix)   # .csv
print(p.parent)   # projekt/dane
print(p.exists()) # True/False

# Odczyt i zapis w jednej linii
txt = p.read_text(encoding='utf-8')
p.write_text('nowa treść')
```

```
# Przeszukiwanie katalogów
for f in Path('.').glob('**/*.py'):
    print(f)

# Tworzenie katalogów
Path('a/b/c').mkdir(
    parents=True, exist_ok=True)

# Iteracja po katalogu
for item in Path('.').iterdir():
    if item.is_file():
        print(item.stat().st_size)
```



pathlib używa operatorów / do łączenia ścieżek — czytelniejsze niż os.path.join() i działa na każdym OS.

CSV i JSON

Strukturalne formaty danych

Moduł csv — odczyt i zapis

```
import csv

# Odczyt CSV
with open('dane.csv', newline='') as f:
    reader = csv.reader(f)
    header = next(reader)
    for row in reader:
        print(row) # lista stringów

# Zapis CSV
with open('w.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Imię', 'Wiek'])
```

DictReader / DictWriter:

```
# Wiersze jako słowniki
with open('dane.csv', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Imię'], row['Wiek'])

# Zapis ze słowników
fields = ['Imię', 'Wiek']
with open('w.csv', 'w', newline='') as f:
    w = csv.DictWriter(f, fields)
    w.writeheader()
    w.writerow({'Imię': 'Jan',
                'Wiek': 30})
```

UTF-8 i kodowanie znaków

```
# Zawsze podawaj encoding!  
with open('pl.txt', 'r',  
        encoding='utf-8') as f:  
    tekst = f.read()  
  
# ord(), chr()  
print(ord('ą')) # 261  
print(chr(9760)) # ☞
```

Dlaczego encoding='utf-8' jest ważne?

Domyślne kodowanie zależy od OS
(Windows: cp1250/cp1252).
Bez jawnego podania możesz dostać
UnicodeDecodeError.

Python 3.15 zmieni domyślne
na UTF-8 (PEP 686).

Obsługa błędów kodowania:

<code>errors='strict'</code>	UnicodeDecodeError (domyślne)
<code>errors='ignore'</code>	pomija nieznane bajty
<code>errors='replace'</code>	wstawia ☐ za nieznane bajty
<code>errors='backslashreplace'</code>	wstawia \xNN

Moduł json

JavaScript Object Notation — uniwersalny format wymiany danych

```
import json

# Python → JSON (serializacja)
data = {'imie': 'Anna', 'wiek': 25,
        'hobby': ['Python', 'jazz']}
json_str = json.dumps(data,
                      ensure_ascii=False, indent=2)

# JSON → Python (deserializacja)
obj = json.loads(json_str)

# Plik → JSON / JSON → plik
with open('d.json', 'w') as f:
    json.dump(data, f, indent=2)
with open('d.json') as f:
    loaded = json.load(f)
```

Mapowanie typów Python ↔ JSON:

dict ↔ object {}

list/tuple ↔ array []

str ↔ string

int/float ↔ number

True/False ↔ true/false

None ↔ null

Protokół iteracji

`__iter__`, `__next__`, `StopIteration`

Iterable vs Iterator

Iterable (obiekt iterowalny)

Posiada metodę `__iter__()` zwracającą iterator.

Można po nim iterować wielokrotnie.

Przykłady: list, tuple, str, dict, set, range

Iterator

Posiada `__iter__()` + `__next__()`.

Zwraca kolejne elementy.

Po wyczerpaniu: `StopIteration`.

Jednorazowe przejście!

Co robi pętla for?

```
# for x in [1, 2, 3]: print(x)
# to w rzeczywistości:
it = iter([1, 2, 3]) # __iter__()
while True:
    try:
        x = next(it) # __next__()
        print(x)
    except StopIteration:
        break
```

`iter()` wywołuje `__iter__()`

`next()` wywołuje `__next__()`

**Każdy iterator jest iterable,
ale nie każdy iterable
jest iteratorem!**

Własny iterator — klasa

```
class Odliczanie:
    """Iterator odliczający od n do 1."""
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self # iterator zwraca sam siebie

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

# Użycie:
for n in Odliczanie(5):
    print(n, end=' ') # 5 4 3 2 1
```

Kluczowe zasady:

`__iter__()` zwraca `self`

`__next__()` zwraca kolejną wartość

raise StopIteration kończy

Stan przechowywany w `self`

💡 Iteratory mogą być nieskończone! np. `itertools.count()` — `list()` na nieskończonym iteratorze zawiesi program.

iter() z dwoma argumentami

iter(callable, sentinel) — wywołuje callable aż zwróci sentinel

```
# Czytanie linii aż do pustej
with open('dane.txt') as f:
    for line in iter(f.readline, ''):
        print(line.strip())

# Losowanie aż do trafienia 7
import random
for n in iter(
    lambda: random.randint(1,10), 7):
    print(n, end= ' ')
```

Dwa tryby iter():

iter(iterable)

→ standardowy iterator

iter(callable, sentinel)

→ wywołuje callable() w pętli,
aż wartość == sentinel
wtedy → StopIteration

Praktyczny przykład — odczyt bloków binarnych:

```
# Czytanie pliku binarnego w blokach po 4096 bajtów
with open('duzy.bin', 'rb') as f:
    for chunk in iter(lambda: f.read(4096), b''):
        process(chunk) # koniec gdy read() zwróci b''
```

Generatory i yield

Leniwa ewaluacja — wartości na żądanie

Funkcja generatora — yield

```
def countdown(n):  
    """Generator odliczający."""  
    while n > 0:  
        yield n      # "zamraża" funkcję  
        n -= 1       # wznowia po next()  
  
gen = countdown(3)  
print(next(gen)) # 3  
print(next(gen)) # 2  
print(next(gen)) # 1  
# next(gen) → StopIteration
```

Jak działa yield?

1. Wywołanie funkcji NIE uruchamia kodu — zwraca obiekt generatora.
2. next() uruchamia kod do yield.
3. yield zwraca wartość i zamraża stan.
4. Kolejne next() wznowia od yield.
5. Koniec funkcji → StopIteration.

Generator vs lista — pamięć:

```
# Lista – wszystko w pamięci  
kwadraty = [x**2 for x in range(10**7)] # ~80 MB!  
  
# Generator – jeden element na raz  
kwadraty_gen = (x**2 for x in range(10**7)) # ~120 B!
```

✓ Generatory są leniwe (lazy):
produkują wartości na żądanie.

Idealne gdy: duże dane,
potok przetwarzania, sekwencje
nieskończone.

Wyrażenia generatorowe

Składnia jak list comprehension, ale z nawiasami okrągłymi

```
# List comprehension → lista
nums = [x**2 for x in range(10)]

# Generator expression → generator
nums_gen = (x**2 for x in range(10))
print(type(nums_gen)) # <generator>

# Jako argument funkcji
sum(x**2 for x in range(10))
any(len(w)>5 for w in words)
','.join(str(n) for n in range(5))
```

Pipeline generatorów:

```
# Każdy krok jest leniwy
lines = (line.strip()
         for line in open('server.log'))
errors = (line for line in lines
          if 'ERROR' in line)
fields = (line.split('|')
          for line in errors)
times = (f[0] for f in fields)

for t in times:
    print(t)
# przetwarzanie strumieniowe,
# linia po linii!
```

send(), throw(), close()

Komunikacja z generatorem w obie strony

```
def accumulator():
    total = 0
    while True:
        value = yield total # yield zwraca,
        if value is None:    # send() wstrzykuje
            break
        total += value

acc = accumulator()
next(acc)           # inicjalizacja → 0
acc.send(10)        # total=10, yield → 10
acc.send(20)        # total=30, yield → 30
acc.send(None)      # break → StopIteration
```

Metody generatora:

send(value)

Wysła wartość do yield.

throw(exc_type)

Wstrzykuje wyjątek w yield.

close()

Wysła GeneratorExit.

💡 **send()** zamienia generator w korutynę (coroutine) — to fundament asyncio.

yield from

Delegacja do subgeneratora

yield from — delegacja iteracji

Bez yield from:

```
def chain_old(*iterables):  
    for it in iterables:  
        for item in it:  
            yield item # podwójna pętla
```

Z yield from:

```
def chain_new(*iterables):  
    for it in iterables:  
        yield from it # elegancko!  
  
list(chain_new([1,2], 'AB', (7,8)))  
# [1, 2, 'A', 'B', 7, 8]
```

Rekurencyjne generatory:

```
def flatten(nested):  
    for item in nested:  
        if isinstance(item, (list, tuple)):  
            yield from flatten(item)  
        else:  
            yield item  
  
list(flatten([1,[2,[3,4]],5])) # [1,2,3,4,5]
```

yield from iterable

równa się:

```
for x in iterable:  
    yield x
```

**ale też przekazuje
send() i throw()!**

yield from i dwukierunkowa komunikacja

```
def worker():
    """Subgenerator przetwarzający dane."""
    total = 0
    while True:
        x = yield          # odbierz dane
        if x is None:
            return total   # zwróć wynik
        total += x

def manager():
    """Delegujący generator."""
    result = yield from worker()
    print(f'Suma: {result}')

m = manager()
next(m)          # inicjalizacja
m.send(10)       # trafia do worker
m.send(20)       # trafia do worker
m.send(None)     # worker robi return 30
```

Co robi yield from?

1. Deleguje next() do sub-gen.
2. Deleguje send() do sub-gen.
3. Deleguje throw() do sub-gen.
4. Przechwytuje return jako wartość yield from.
5. Propaguje StopIteration.

⚠ yield from to pełny „most” między wywołującym a subgeneratorem. W asyncio to fundament.

itertools

Zaawansowane narzędzia iteracji z biblioteki standardowej

Nieskończone iteratory

```
from itertools import count, cycle, repeat

# count(start=0, step=1)
for i in count(10, 2): # 10,12,14,..
    if i > 20: break
    print(i, end=' ')
# 10 12 14 16 18 20

# cycle('ABCD') → A B C D A B..
c = cycle(['♠', '♥', '♦'])
for _ in range(7):
    print(next(c), end=' ')
# ♠ ♥ ♦ ♠ ♥ ♦ ♠
```

```
# repeat(obj, times=None)
list(repeat('X', 5))
# ['X', 'X', 'X', 'X', 'X']

# Jako argument map:
list(map(pow, range(5), repeat(2)))
# [0, 1, 4, 9, 16]
```

⚠ Nieskończone iteratory!

Nigdy nie rób:

```
list(count())
list(cycle([1,2]))
```

Zawsze ogranicz: islice,
takewhile, lub break.



itertools zwraca iteratory (leniwe) — zużycie pamięci $O(1)$, niezależnie od rozmiaru danych.

chain, islice, compress

chain / islice:

```
from itertools import chain, islice

list(chain([1,2], 'AB', (7,)))
# [1, 2, 'A', 'B', 7]

list(islice(count(), 5, 10))
# [5, 6, 7, 8, 9]

list(islice('ABCDEFGH', 2, 6))
# ['C', 'D', 'E', 'F']
```

compress / takewhile / dropwhile:

```
from itertools import compress
list(compress('ABCDEF',
              [1,0,1,0,1,1])) # ['A','C','E','F']

from itertools import takewhile
list(takewhile(lambda x: x<5,
               [1,3,5,2,1])) # [1, 3]

from itertools import dropwhile
list(dropwhile(lambda x: x<5,
                [1,3,5,2,1])) # [5, 2, 1]
```

Praktyka — odczyt N pierwszych linii pliku:

```
# Jak head -n 10 w Unix
with open('duzy_log.txt') as f:
    for line in islice(f, 10): # tylko 10 linii, bez wczytywania całości
        print(line, end='')
```

groupby — grupowanie elementów

```
from itertools import groupby

# UWAGA: dane MUSZĄ być posortowane wg klucza!
data = [('A',1),('A',2),('B',3),('B',4),('C',5)]

for key, group in groupby(data,
                           key=lambda x: x[0]):
    items = list(group)
    print(f'{key}: {items}')
# A: [('A',1), ('A',2)]
# B: [('B',3), ('B',4)]
# C: [('C',5)]
```

 **groupby wymaga posortowanych danych!**

```
# Nie posortowane:
data = [1,1,2,1,2,2]
for k, g in groupby(data):
    print(k, list(g))
# 1 [1,1], 2 [2],
# 1 [1], 2 [2,2]
# → 4 grupy zamiast 2!
```

Praktyczny przykład — grupowanie logów:

```
logs = sorted(logs, key=lambda e: e['level']) # najpierw sortowanie!
for level, entries in groupby(logs, key=lambda e: e['level']):
    entry_list = list(entries) # UWAGA: group to iterator, zużywa się!
    print(f'{level}: {len(entry_list)} wpisów')
```

Kombinatoryka — product, permutations, combinations

```
from itertools import product

# Iloczyn kartezjański
list(product('AB', [1,2]))
# [('A',1),('A',2),('B',1),('B',2)]

# Zastępuje zagnieżdżone pętle!
for s, r in product('AKQJ', [1,2]):
    print(s, r)

from itertools import permutations
list(permutations('ABC', 2))
# [('A','B'),('A','C'),('B','A'),...]
```

```
from itertools import combinations
list(combinations('ABCD', 2))
# [('A','B'),('A','C'),('A','D'),...]

from itertools import (
    combinations_with_replacement)
list(combinations_with_replacement(
    'ABC', 2))
```

Porównanie:

product	→ n^k elementów
permutations	→ $n!/(n-k)!$
combinations	→ $n!/k!(n-k)!$

Przykład — brute-force hasła (edukacyjny!):

```
for combo in product('0123456789', repeat=4): # 10^4 = 10000 kombinacji
    pin = ''.join(combo)
    if check_pin(pin): break
```

starmap, tee, accumulate

starmap:

```
from itertools import starmap

# starmap – rozpakowuje argumenty
list(starmap(pow,
              [(2,3),(3,2),(10,3)]))
# [8, 9, 1000]

list(starmap(max, [(5,3),(1,8)]))
# [5, 8]
```

tee:

```
from itertools import tee

nums = iter(range(5))
a, b = tee(nums, 2) # 2 kopie

list(a) # [0, 1, 2, 3, 4]
list(b) # [0, 1, 2, 3, 4]
# oryginał nums jest zużyty!
```

accumulate:

```
from itertools import accumulate
import operator

list(accumulate([1,2,3,4,5]))          # [1, 3, 6, 10, 15] (sumy bieżące)
list(accumulate([3,1,4,1,5], max))      # [3, 3, 4, 4, 5] (max bieżący)
list(accumulate([1,2,3,4,5], operator.mul)) # [1, 2, 6, 24, 120] (silnie!)
list(accumulate([1,2,3], initial=100))  # [100, 101, 103, 106]
```



accumulate to funkcjonalny odpowiednik "running total" — analiza danych, przetwarzanie sygnałów.

Nowe w itertools — pairwise, batched

pairwise (3.10+):

```
from itertools import pairwise

list(pairwise('ABCDE'))
# [('A', 'B'), ('B', 'C'),
#  ('C', 'D'), ('D', 'E')]

# Przydatne: różnice kolejnych
data = [10, 13, 17, 22]
diffs = [b-a for a,b in pairwise(data)]
# [3, 4, 5]
```

batched (3.12+):

```
from itertools import batched

list(batched('ABCDEFGH', 3))
# [('A', 'B', 'C'), ('D', 'E', 'F'),
#  ('G',)]

# Przetwarzanie w paczkach:
for batch in batched(big_data, 100):
    send_to_api(batch)
```

Wcześniejsze podejście (przed batched):

```
# Popularna receptura "chunked" sprzed Python 3.12
def chunked(iterable, n):
    it = iter(iterable)
    while chunk := list(islice(it, n)): # walrus operator!
        yield chunk
list(chunked(range(7), 3)) # [[0,1,2], [3,4,5], [6]]
```

Itertools w praktyce — pipeline przetwarzania

```
"""Pipeline: analiza logów serwera z użyciem itertools."""
from itertools import chain, groupby, islice, dropwhile
from pathlib import Path
import csv

# 1. Zbierz logi z wielu plików (leniwie!)
log_files = Path('logs/').glob('*.log')
all_lines = chain.from_iterable(open(f) for f in log_files)

# 2. Pomiń nagłówki (linie zaczynające się od #)
data_lines = dropwhile(lambda l: l.startswith('#'), all_lines)

# 3. Parsuj jako CSV
reader = csv.reader(data_lines, delimiter='|')

# 4. Filtruj błędy
errors = (row for row in reader if row[1].strip() == 'ERROR')

# 5. Pobierz pierwszych 100
for row in islice(errors, 100):
    print(f'{row[0]} | {row[2]}')

# Cały pipeline: O(1) pamięci, niezależnie od rozmiaru logów!
```


contextlib

Dekoratory i narzędzia menedżerów kontekstu

@contextmanager — CM z generatora

Twórz menedżer kontekstu bez klasy z `__enter__`/`__exit__`

```
from contextlib import contextmanager

@contextmanager
def timer(label):
    import time
    start = time.perf_counter()
    yield # tu wykonuje się blok with
    elapsed = time.perf_counter()-start
    print(f'{label}: {elapsed:.3f}s')

with timer('Sortowanie'):
    sorted(range(10**6, 0, -1))
# Sortowanie: 0.142s
```

Schemat @contextmanager:

```
@contextmanager
def my_cm():
    resource = acquire()
    try:
        yield resource # as ...
    finally:
        release(resource)
```

Przykład z tymczasowym katalogiem:

```
@contextmanager
def temp_dir():
    d = Path(tempfile.mkdtemp())
    try:
        yield d
    finally:
        shutil.rmtree(d)
```

suppress, redirect, ExitStack

suppress:

```
from contextlib import suppress

# Zamiast try/except/pass:
with suppress(FileNotFoundError):
    os.remove('tmp.txt')
```

redirect_stdout:

```
from contextlib import redirect_stdout
import io
buf = io.StringIO()
with redirect_stdout(buf):
    print('To trafi do bufora')
```

ExitStack — dynamiczna liczba CM:

```
from contextlib import ExitStack

filenames = ['a.txt', 'b.txt', 'c.txt']

with ExitStack() as stack:
    files = [
        stack.enter_context(open(f))
        for f in filenames
    ]
    # Wszystkie otwarte, zamkną się auto
```

ExitStack pozwala:

- Otwierać dowolną liczbę zasobów w pętli
- Rejestrować callback'i:

`stack.callback(func)`

- Bezpiecznie zamykać wszystko w kolejności LIFO

chdir i nullcontext

chdir (3.11+):

```
from contextlib import chdir

# Tymczasowa zmiana katalogu
with chdir('/tmp'):
    print(os.getcwd()) # /tmp
print(os.getcwd()) # oryginalny
```

nullcontext:

```
from contextlib import nullcontext

# "Pusty" CM – przydatny warunkowo
cm = (open('f.txt') if need_file
      else nullcontext('default'))
with cm as val:
    process(val)
```

Przykład: warunkowe logowanie:

```
from contextlib import contextmanager, nullcontext

@contextmanager
def log_to_file(path):
    with open(path, 'a') as f:
        yield f

def process(data, log_path=None):
    cm = log_to_file(log_path) if log_path else nullcontext()
    with cm as log:
        result = heavy_computation(data)
        if log:
            log.write(f'Result: {result}\n')
```

Buforowanie i flush

Kiedy dane trafiają na dysk?

```
from time import sleep

# flush=True → natychmiast!
for i in range(10):
    print(i, end=' ', flush=True)
    sleep(0.5)

# Efekt spinnera ASCII
for char in '|/_\\' * 10:
    print(f' {char} ', end='',
          flush=True)
    sleep(0.2)
print('\n', end='', flush=True)
```

```
# Bufor w plikach
f = open('log.txt', 'w',
         buffering=1) # liniowy
f = open('data.bin', 'wb',
         buffering=4096) # 4KB
```

Argumenty print() a I/O:

sep=' '	separator między argumentami
end='\n'	zakończenie wyjścia
file=sys.stdout	strumień docelowy
flush=False	wymuszenie opróżnienia

seek() i tell() — nawigacja w pliku

```
with open('dane.txt', 'r+') as f:
    print(f.tell()) # 0 (początek)
    f.read(5)       # czytaj 5 znaków
    print(f.tell()) # 5
    f.seek(0)       # wróć na początek

# seek(offset, whence)
# whence: 0=początek, 1=bieżąca, 2=koniec

# Odczyt ostatnich 100 bajtów
with open('log.txt', 'rb') as f:
    f.seek(-100, 2) # 100B od końca
    tail = f.read()
```

Przykład — implementacja tail -f:

```
import time
def tail_f(path):
    with open(path) as f:
        f.seek(0, 2) # idź na koniec
        while True:
            line = f.readline()
            if line:
                print(line, end='')
            else:
                time.sleep(0.5)

# Nadpisanie fragmentu
with open('dane.bin', 'r+b') as f:
    f.seek(10) # idź do bajtu 10
    f.write(b'\xFF') # nadpisz 1 bajt
```

Obiekt z metodą write() jako plik

```
class Pisarz:
    """Obiekt zbierający dane z print()."""
    def __init__(self):
        self.dane = []

    def write(self, tekst):
        self.dane.append(tekst)

    def czytaj(self):
        return ''.join(self.dane)

pisarz = Pisarz()
print('Pierwszy tekst.', file=pisarz)
print('Drugi tekst!', file=pisarz)
print(pisarz.czytaj())
```

Duck typing w I/O:

Argument file= w print() nie wymaga prawdziwego pliku — wystarczy obiekt z metodą write(string).

„Jeśli chodzi jak plik i pisze jak plik, to jest plikiem.”

io.StringIO / io.BytesIO:

```
import io
# StringIO – tekstowy plik w pamięci
buf = io.StringIO()
buf.write('Hello ')
buf.write('World')
buf.seek(0)
print(buf.read()) # 'Hello World'
```

itertools — szybka ściągą

Nieskończone	<code>count, cycle, repeat</code>
Łączenie	<code>chain, chain.from_iterable, zip_longest</code>
Filtrowanie	<code>compress, dropwhile, takewhile, filterfalse, islice</code>
Grupowanie	<code>groupby, pairwise, batched</code>
Kombinatoryka	<code>product, permutations, combinations, comb_with_replacement</code>
Transformacja	<code>starmap, accumulate, tee</code>
Receptury	<code>tabulate, consume, nth, quantify, flatten, unique_everseen</code>



Pełna dokumentacja: docs.python.org/3/library/itertools.html — warto przeczytać receptury!

Typowe błędy z iteratorami

Błąd 1: Wielokrotne zużycie iteratora

```
nums = (x**2 for x in range(5))
print(list(nums)) # [0,1,4,9,16]
print(list(nums)) # [] !!! pusty!

# Rozwiązanie: użyj listy lub tee()
from itertools import tee
nums = (x**2 for x in range(5))
a, b = tee(nums, 2)
print(list(a)) # [0,1,4,9,16]
print(list(b)) # [0,1,4,9,16]
```

Błąd 2: Niezamknięty plik

```
# ŹŁE: plik nigdy nie zamknięty
lines = open('f.txt').readlines()
# Wyciek zasobów!

# DOBRZE: with gwarantuje close()
with open('f.txt') as f:
    lines = f.readlines()
```

Błąd 3: groupby bez sortowania

groupby([1,2,1]) → 3 grupy, nie 2!
Zawsze sortuj przed groupby!

Błąd 4: list() na nieskończonym iteratorze

list(count()) → zawiesi program! Użyj islice() lub takewhile() aby ograniczyć liczbę elementów.

Pliki, I/O i protokół iteracji

1	open() i tryby	'r', 'w', 'a', 'x', 'b' — zawsze podawaj encoding='utf-8'
2	with (context manager)	__enter__ / __exit__ — bezpieczne zarządzanie zasobami
3	pathlib	nowoczesne API ścieżek: Path / operatorzy / glob
4	csv i json	csv.DictReader, json.dumps/loads — standard wymiany danych
5	Iterable vs Iterator	__iter__() zwraca iterator, __next__() daje elementy
6	Generatory i yield	leniwa ewaluacja — O(1) pamięci, pipeline'y
7	yield from	delegacja do subgeneratora z pełną komunikacją
8	itertools	chain, groupby, product, starmap, accumulate, tee, batched
9	contextlib	@contextmanager, suppress, ExitStack, nullcontext